

Task 1 - XML Schema

1.1

A well-formed XML document must follow these two rules:

- First, it has to have exactly one top-level (root) element that contains all other elements.
- Second, it has to be nested properly. This means every start tag `<tag>` must have a matching closing tag `</tag>`, and different tags must not overlap (e.g. not allowed: `<a><b></a></b>`).

In this case the document has just one top-level element `<tns:Orders>`. It is also properly nested, as every tag has a matching closing tag and no tags overlap. The document is therefore well-formed.

1.2

- Lines 17–21: The DeliveryAddress has the type `tAddress`, which requires the element `<StreetNumber>`. Since the elements of an `xs:all` group have a default `minOccurs` of 1, `StreetNumber` is mandatory but missing here.
- Line 26: The Quantity is 10, but the schema restricts Quantity with `maxExclusive` value 10, so the maximum allowed value is 9.
- Line 41: The BillingAddress also has the type `tAddress`, so it is required to have an `addressID`.
- Line 44: The ZipCode of a `tAddress` type has to consist of exactly five digits between 0 and 9. This one has only four.
- Lines 48–52: The child elements of the Item are in the wrong order. The type `tOrderItem` is defined as an `xs:sequence` (Name, Size, Quantity), but here Quantity appears first.
- Line 53: The itemID has to be unique, but the ID I000328 has already been used in line 48.

1.3

No, one of the criteria for a document to be valid is, that it is well-formed.

Task 2 - XQuery

2.1

```
for $hotel in doc("hotels.xml")//Hotel
let $resCount := count($hotel//Reservation)
return
  <output>
    {concat($hotel/Name, " ", $hotel/Address/TownName, " ", $resCount)}
  </output>
```

2.2

```
for $hotel in doc("hotels.xml")//Hotel,
  $guest in $hotel/Guests/Guest,
  $res in $hotel//Reservation[@guestNumber = $guest/@guestNumber]
return <output>{ $guest/Name/text() } { $res }</output>
```

2.3

```
for $hotel in doc("hotels.xml")//Hotel
let $shortNames := $hotel//Guest/Name[string-length(.) < 11]
where exists($shortNames)
return
  <output>
    {concat($hotel/@hotelNumber, " ", string-join($shortNames, ", "))}
  </output>
```

2.4

```
for $hotel in doc("hotels.xml")//Hotel,
  $res in $hotel//Reservation[RoomType/BasicRoom/MiniBar = "false"],
  $guest in $hotel/Guests/Guest[@guestNumber = $res/@guestNumber]
order by $guest/Address/StreetAndHouseNumber ascending
return <output>{ concat($guest/@guestNumber, " ",
  $guest/Address/StreetAndHouseNumber) }</output>
```

Task 3 - DTD and XML Schema

3.1

The DTD:

```
<!ELEMENT machines (machine+)>
<!ELEMENT machine (machine-name, maintenance-engineer, incidents)>
<!ATTLIST machine machine-number ID #REQUIRED>
<!ELEMENT machine-name (#PCDATA)>
<!ELEMENT maintenance-engineer (#PCDATA)>
<!ELEMENT incidents (incident+)>
<!ELEMENT incident (incident-message, incident-code, incident-date, machine-t
```

```
<!ELEMENT incident-message (#PCDATA)>
<!ELEMENT incident-code (#PCDATA)>
<!ELEMENT incident-date (#PCDATA)>
<!ELEMENT machine-temperature (#PCDATA)>
```

An example document that uses this DTD:

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE machines [
  <!ELEMENT machines (machine+)>
  <!ELEMENT machine (machine-name, maintenance-engineer, incidents)>
  <!ATTLIST machine
    machine-number ID #REQUIRED>
  <!ELEMENT incidents (incident+)>
  <!ELEMENT incident (incident-message, incident-code,
    incident-date, machine-temperature?)>
  <!ELEMENT machine-name (#PCDATA)>
  <!ELEMENT maintenance-engineer (#PCDATA)>
  <!ELEMENT incident-message (#PCDATA)>
  <!ELEMENT incident-code (#PCDATA)>
  <!ELEMENT incident-date (#PCDATA)>
  <!ELEMENT machine-temperature (#PCDATA)>
]>
```

There are some requirements, that a DTD-document can't fulfill:

- A machine-number must have exactly 5 characters. The DTD-document can't restrict the length of the data.
- A machine-temperature must contain a floating point number. The DTD-document can't restrict text to floating point numbers.
- Using the ID type to enforce uniqueness of machine-number restricts the value to a valid XML Name, which must not start with a digit. A purely numeric 5-character machine-number is therefore not expressible in a DTD. Likewise, incident-date cannot be validated as a date, only as #PCDATA.

3.2

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="machines">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="machine" type="machineType" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
    <xs:unique name="uniqueMachineNumber">
      <xs:selector xpath="machine"/>
    </xs:unique>
  </xs:element>
</xs:schema>
```

```
<xs:field xpath="@machine-number" />
</xs:unique>
</xs:element>

<xs:complexType name="machineType">
  <xs:sequence>
    <xs:element name="machine-name" type="xs:string" />
    <xs:element name="maintenance-engineer" type="xs:string" />
    <xs:element name="incidents" type="incidentsType" />
  </xs:sequence>
  <xs:attribute name="machine-number" type="machineNumberType" use="required" />
</xs:complexType>

<xs:complexType name="incidentsType">
  <xs:sequence>
    <xs:element name="incident" type="incidentType" maxOccurs="unbounded" />
  </xs:sequence>
</xs:complexType>

<xs:complexType name="incidentType">
  <xs:sequence>
    <xs:element name="incident-message" type="xs:string" />
    <xs:element name="incident-code" type="xs:string" />
    <xs:element name="incident-date" type="xs:date" />
    <xs:element name="machine-temperature" type="xs:float" minOccurs="0" />
  </xs:sequence>
</xs:complexType>

<!-- machine-number: exactly 5 characters, unique via xs:unique above -->
<xs:simpleType name="machineNumberType">
  <xs:restriction base="xs:string">
    <xs:length value="5" />
  </xs:restriction>
</xs:simpleType>

</xs:schema>
```

3.3

- One advantage is, that with a XML-schema, there is a bigger differentiation between simple data types. In DTD you only have #PCDATA, while you have integer, strings, dates, ... in a XML-schema.
- Another advantage would be the wider range for an XML-schema to have more complex structures due to things like minOccurs, maxOccurs, ...